

Lecture 10: Classification

ML/AI for Engineering Applications
ENV 6932

Sara Behdad
Fall 2025

Classification

- Decision Tree
- Random Forest
- Support Vector Machine

The Naive Bayes algorithm

- A classification algorithm based on Bayes rule and a set of conditional independence assumptions.
- The goal: learning $P(Y|X)$ where $X = \langle X_1 \dots, X_n \rangle$,
- Assumption that each X_i is conditionally independent of each of the other X_k s given Y , and also independent of each subset of the other X_k 's given Y .
- The probability that Y will take on its k th possible value, according to Bayes rule:

$$P(Y = y_k | X_1 \dots X_n) = \frac{P(Y = y_k)P(X_1 \dots X_n | Y = y_k)}{\sum_j P(Y = y_j)P(X_1 \dots X_n | Y = y_j)}$$

The Naive Bayes algorithm

- Naïve Bayes Assumption:

$$P(\mathbf{x}|y) = \prod_i P(x_i|y)$$

- Therefore, the fundamental equation for the Naive Bayes classifier is:

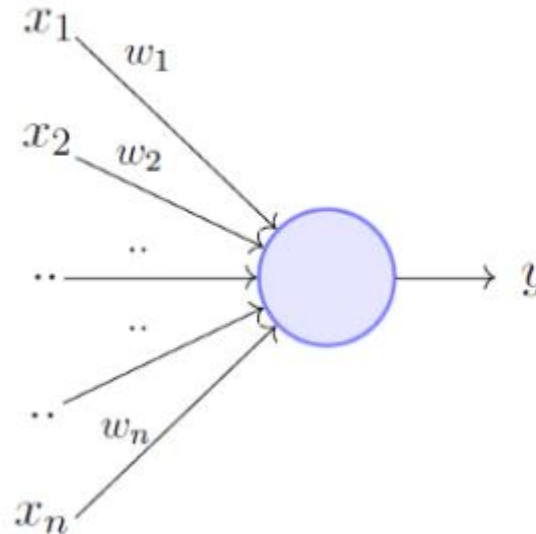
$$P(Y = y_k | X_1 \dots X_n) = \frac{P(Y = y_k) \prod_i P(X_i | Y = y_k)}{\sum_j P(Y = y_j) \prod_i P(X_i | Y = y_j)}$$

- which simplifies to the following (since the denominator does not depend on y_k).

$$Y \leftarrow \arg \max_{y_k} P(Y = y_k) \prod_i P(X_i | Y = y_k)$$

Perceptron Learning Algorithm

- Originally proposed by Frank Rosenblatt in 1943, Later developed and carefully analyzed by Minsky and Papert in 1969.
- The perceptron model takes an input, aggregates it (weighted sum) and returns 1 only if the aggregated sum is more than some threshold else returns 0.



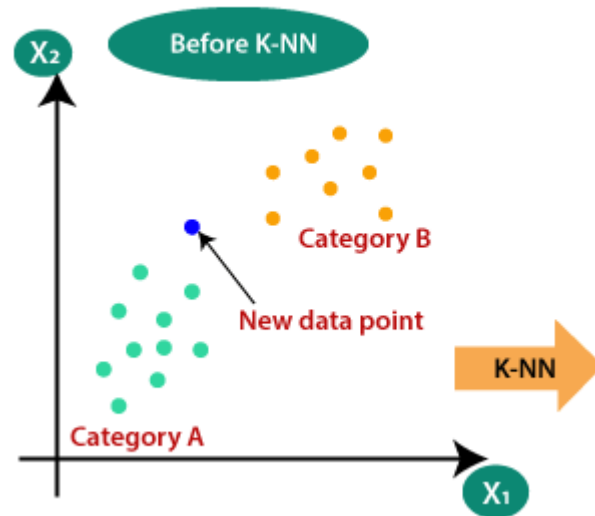
$$y = 1 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i \geq \theta$$
$$= 0 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i < \theta$$

Rewriting the above,

$$y = 1 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i - \theta \geq 0$$
$$= 0 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i - \theta < 0$$

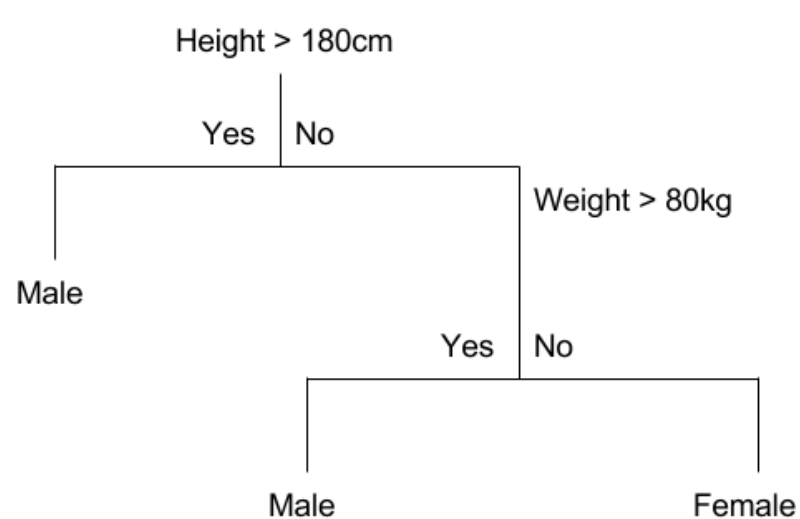
K-Nearest Neighbors

- Example: we have two categories, i.e., Category A and Category B. What is the category of a new data point?

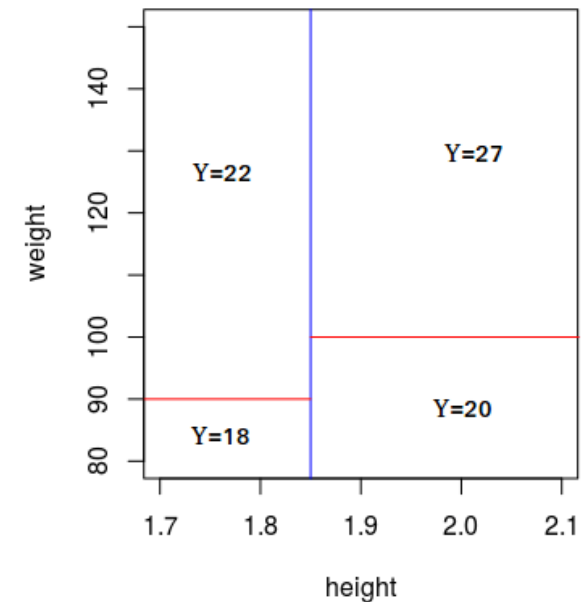
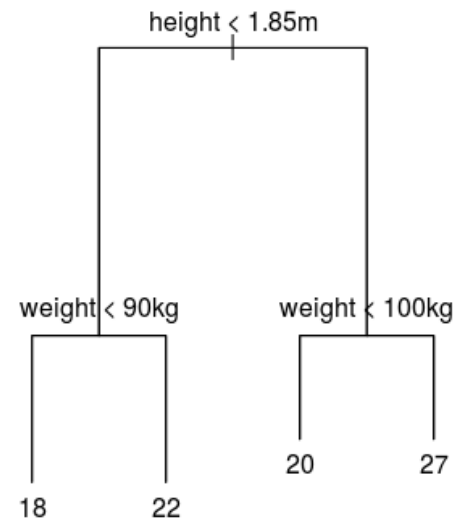


Two Main Types of Decision Trees

- **Classification trees** (Yes/No types)
- **Regression trees** (Continuous data types)



Example of a Classification Tree



Example of Regression Tree

Ways to Split a Decision Tree

- Continuous Target Variable
 - Reduction in Variance
- Categorical Target Variable
 - Entropy (a measure of randomness)
 - Gini Impurity
 - Information Gain
 - Chi-Square

Information Gain

- How to measure the reduction in disorder?
- A metric to measure the reduction of disorder in our target variable/class given additional information (features/independent variables) about it.
- This is called Information Gain:

$$IG(Y, X) = E(Y) - E(Y|X)$$

Information Gain from X on Y

- Decision trees use entropy and information gain to decide on what features to split their nodes on.

Splitting Method #1: Reduction in Variance

- A method used when the target variable is continuous, i.e., regression problems.
- It uses variance as a measure for deciding the feature on which node is split into child nodes.

$$Variance = \frac{\sum (X - \mu)^2}{N}$$

Splitting Method #2: Information Gain

- For the case of categorical target variable.
- Entropy is used for calculating the randomness or purity of a node.
- **Lower the value of entropy, higher is the purity of the node.**

$$Entropy = - \sum_{i=1}^n p_i \log_2 p_i$$

$$Information\ Gain = 1 - Entropy$$

Splitting Method #3: Gini Impurity

- The target variable is categorical.
- Gini is the probability of correctly labeling a randomly chosen element if it was randomly labeled according to the distribution of labels in the node.

$$\text{Gini Impurity} = 1 - \text{Gini}$$

$$\text{Gini} = \sum_{i=1}^n p_i^2$$

$$\text{Gini Impurity} = 1 - \sum_{i=1}^n p_i^2$$

Splitting Method #4: Chi-Square

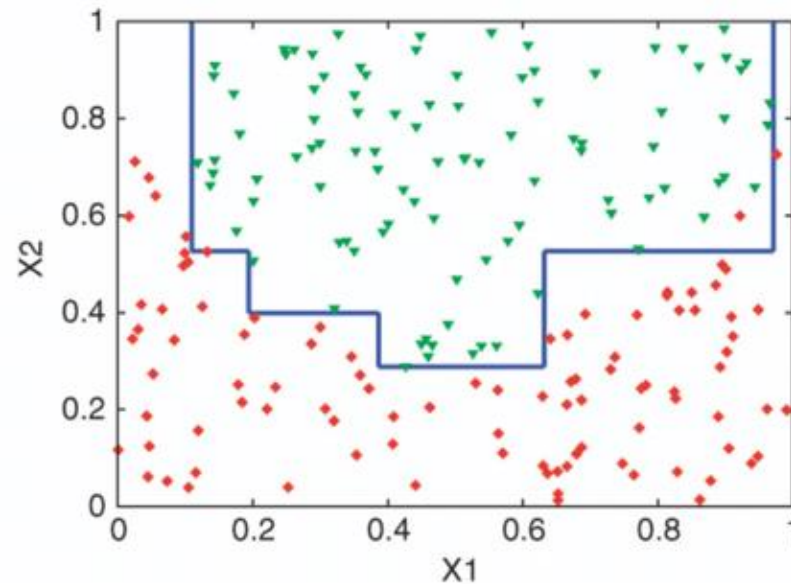
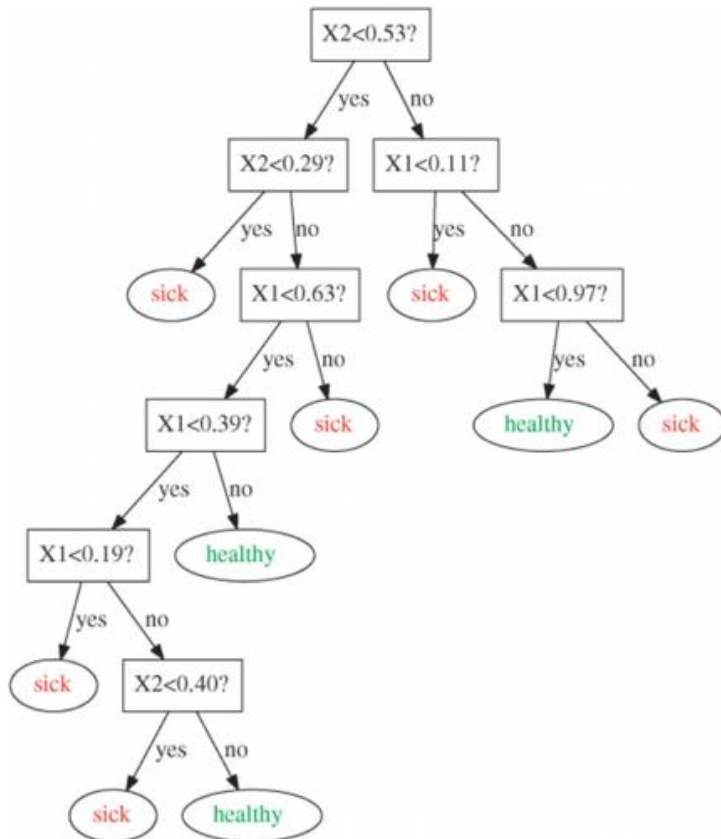
- Used for categorical target values.
- It can create two or more than two splits.
- The statistical significance of differences between the parent node and child nodes.

$$Chi-Square = \sqrt{\frac{(Actual - Expected)^2}{Expected}}$$

Where, the *Expected* is the expected value for a class in a child node based on the distribution of classes in the parent node, and *Actual* is the actual value for a class in a child node.

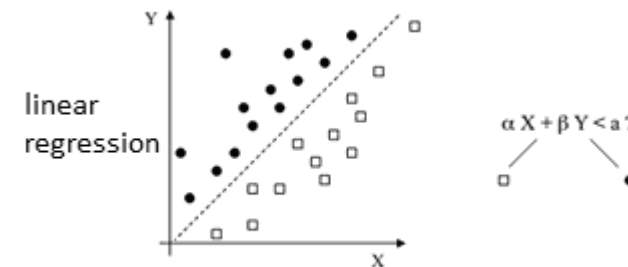
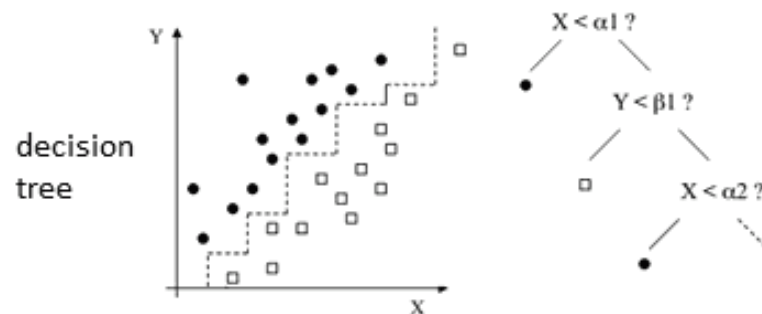
Disadvantage of a Decision Tree

- Tends to overfit data (high variance and false predictions in the test data).



How does DT make decision?

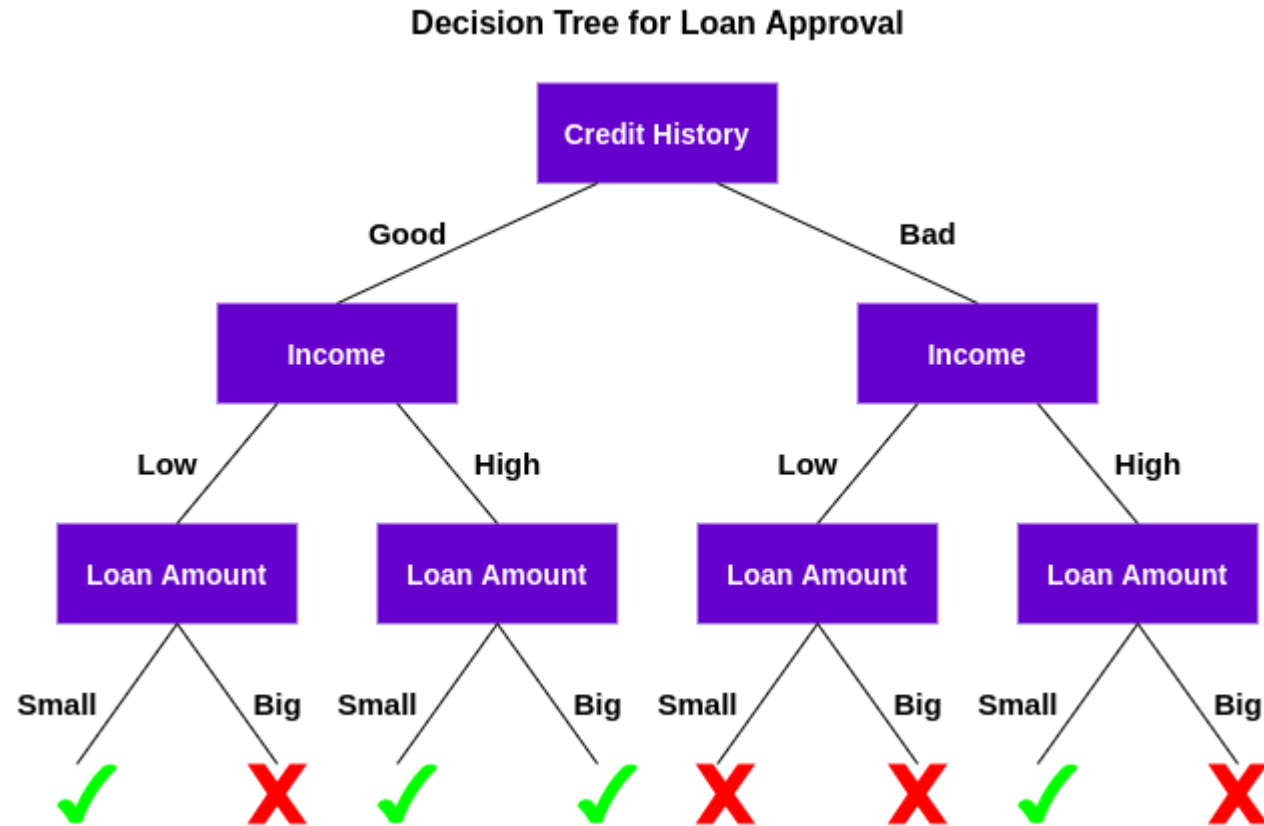
- The decision tree approaches the training data through the sequence of questions rather than understanding the general pattern of the data as linear regression does.



Random Forest Classifier

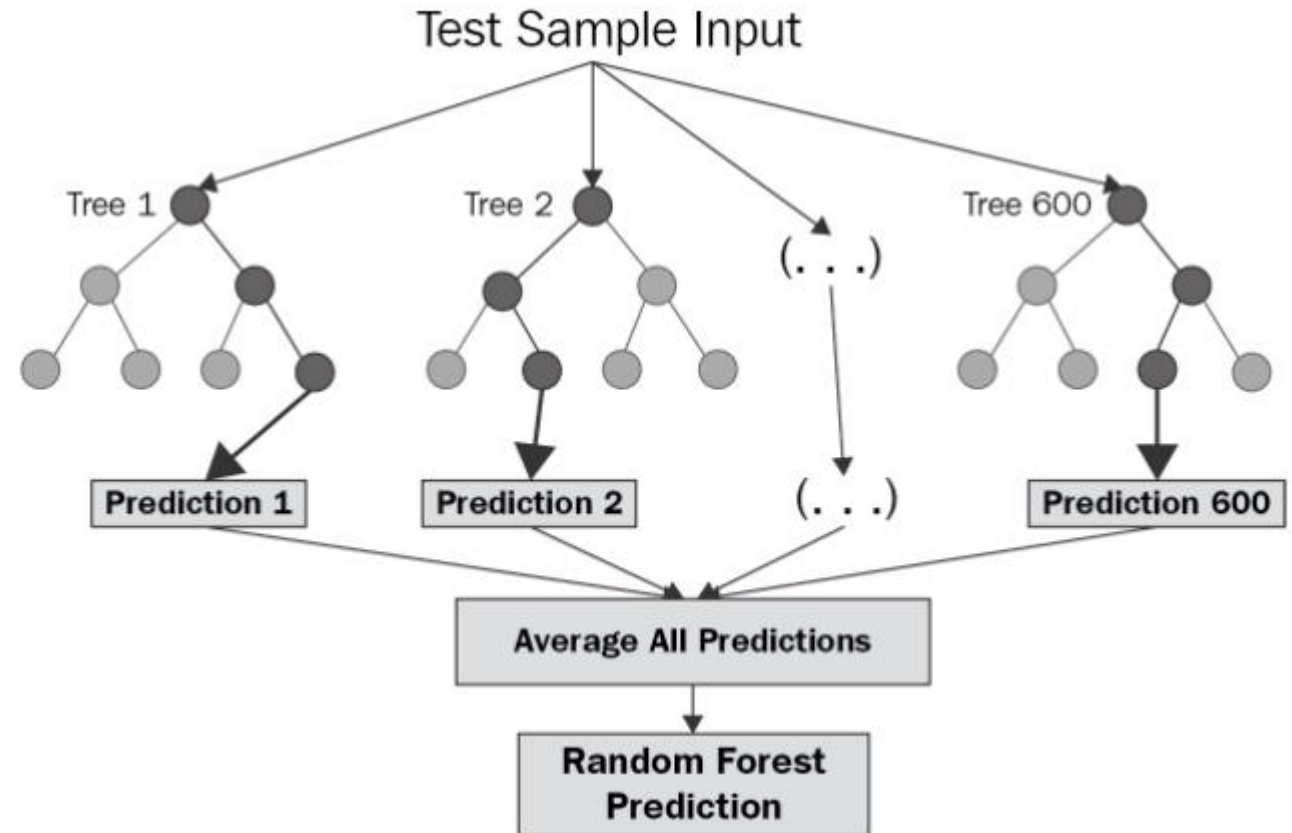
- The difference between a decision tree and a random forest model.
- **Example:** approving a small loan by the bank
- **Scenario 1:** The bank first checks the credit history and discovers that they haven't re-paid the older loan yet. Hence, the bank rejects the application and lose the chance of lending money.
- **Scenario 2:** Sometimes it checks for credit history first, and sometimes for customer's financial condition and loan amount first. Then, combines results from these multiple decision-making processes.

Decision Tree



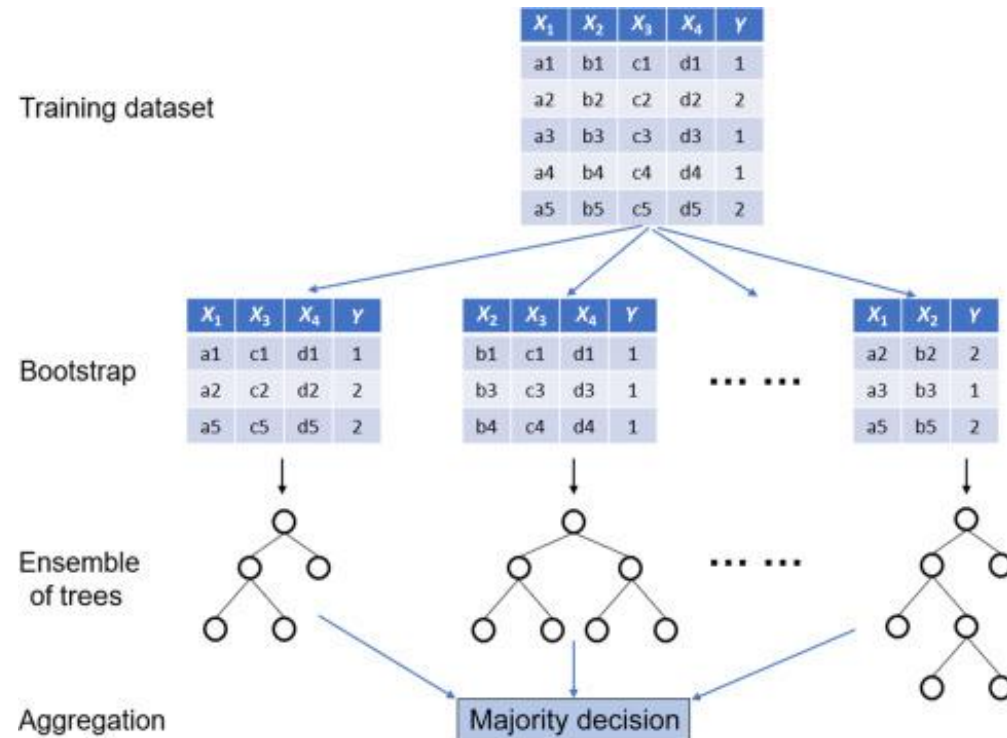
Random Forest Uses Ensemble Learning

- The Random Forest Algorithm combines the output of multiple (randomly created) Decision Trees to generate the final output.
- **Ensemble Learning:** The process of merging the output of multiple individual models (also known as weak learners).



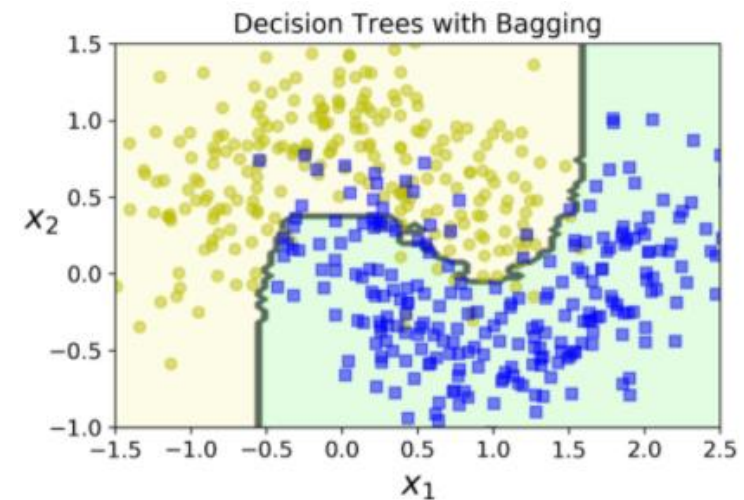
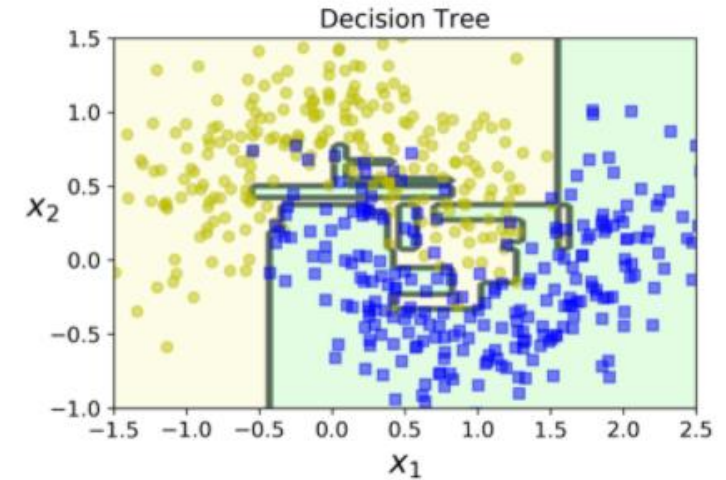
Bootstrapping in RF

- **Bootstrapping:** is a process of creating a subset of data from the original data.
- Bootstrapping ensures that each individual decision tree in the random forest is unique, which reduces the overall variance of the RF classifier.
- The RF classifier aggregates the decisions of individual trees; consequently, RF classifier exhibits good generalization without issues of overfitting.



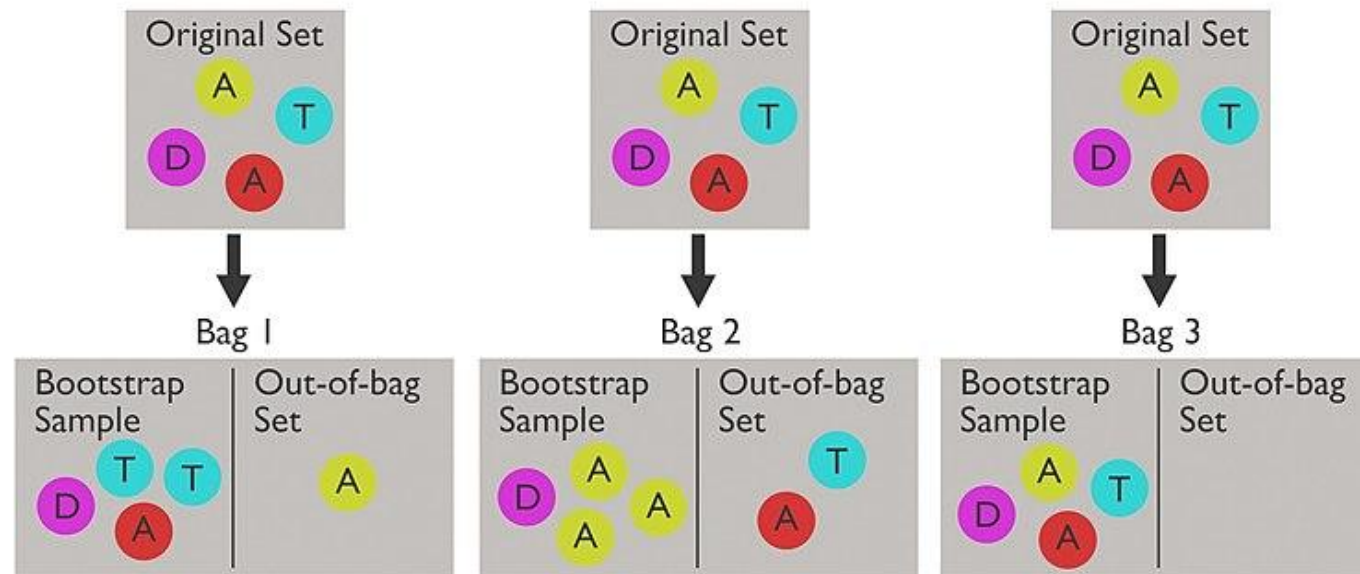
Bootstrapping in RF

- **Bagging:** is a sampling technique subsets of observations from the original dataset will be created, **with replacement**.
- Replacement means we allow the duplication of the data.
- The size of the subsets is the same as the size of the original set.



The Concept of Bagging

- Replacement allows the duplication of the data.
- In bagging, the size of the subsets is the same as the size of the original set.



Summary of Random Forest

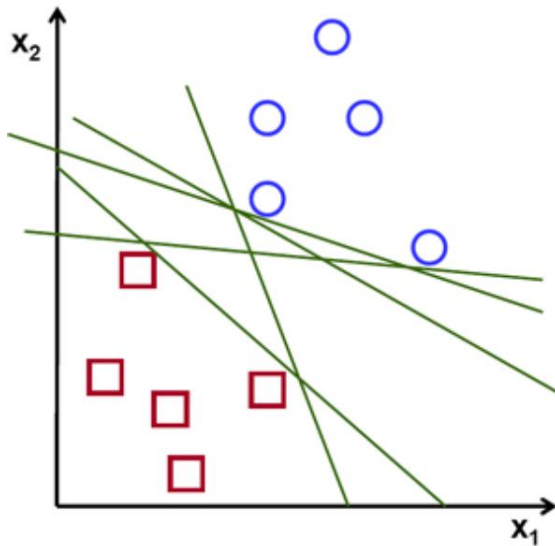
1. Create 1000+ (`n_estimator`) decision trees.
2. Take prediction from each individual decision tree.
3. Combined together and perform a voting processing (aggregation), the maximum vote is chosen as the right prediction.

Support Vector Machine

- SVM is preferred by many due to its significant accuracy with less computation power.
- SVM can be used for both regression and classification tasks.
- However, it is widely used in classification objectives.

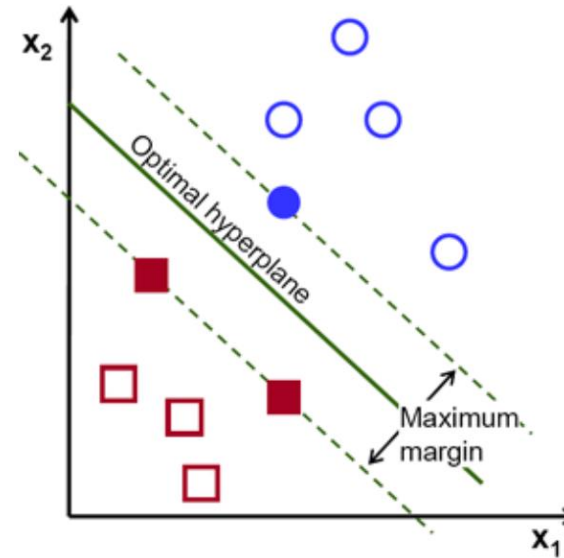
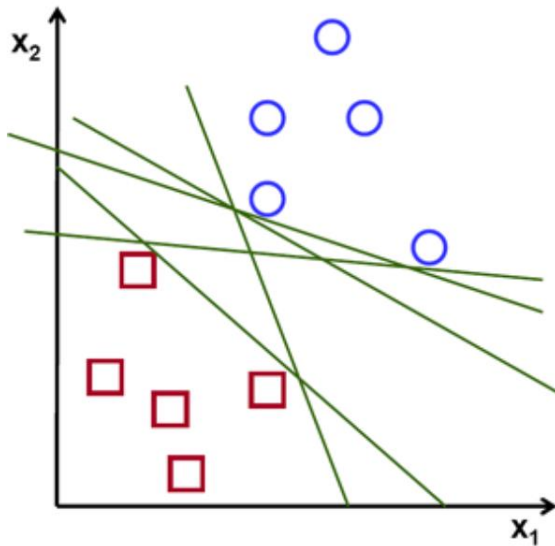
Support Vector Machine

- **Objective:** to find a hyperplane in an N-dimensional space (N is the number of features) that clearly classifies the data points.



Support Vector Machine

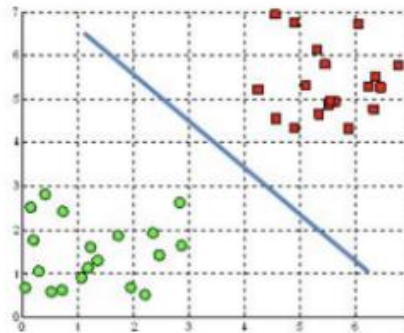
- **Objective:** to find a hyperplane in an N-dimensional space (N is the number of features) that clearly classifies the data points.



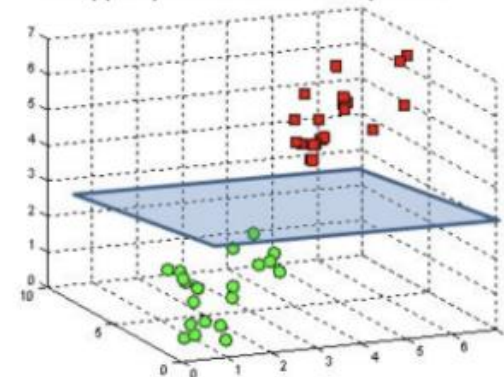
Support Vector Machine

- **Objective:** to find a plane that has the maximum margin, i.e the maximum distance between data points of both classes.
- Maximizing the margin distance $\sim$ future data points can be classified with more confidence.

A hyperplane in $\mathbb{R}^2$ is a line



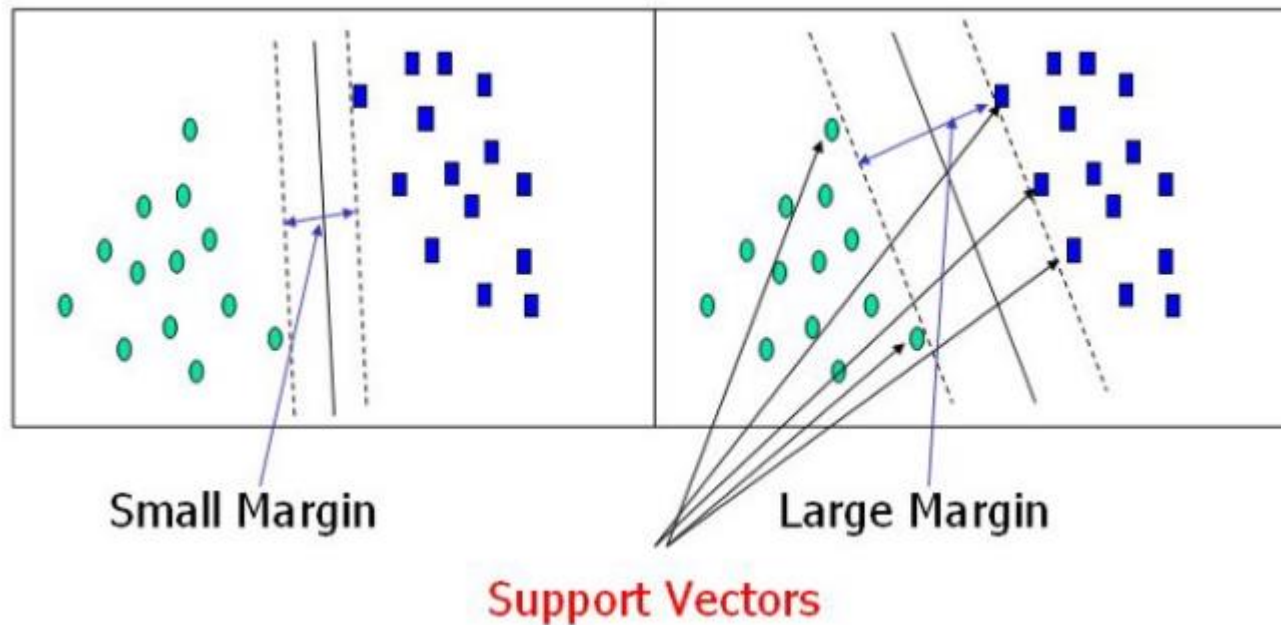
A hyperplane in $\mathbb{R}^3$ is a plane



Hyperplanes in 2D and 3D feature space

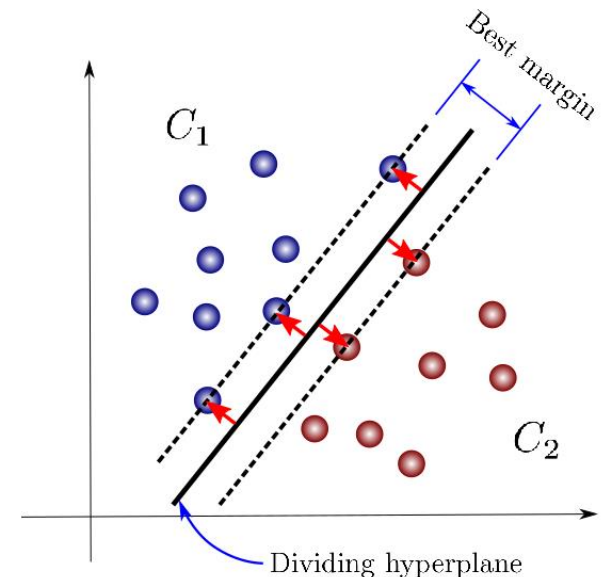
Support Vector Machine

- Support vectors: **data points** that are closer to the hyperplane and influence the position and orientation of the hyperplane.



How to find the dividing hyperplane?

- Two categories, each identified by $C_1 = 1$ and $C_2 = -1$ (a *binary classification problem*).
- Goal: Identify the *margin*.
- *The process is* governed by the distance formed between the closest points and the separating hyperplane.



Deriving the primal problem

- Suppose the dataset is composed of N points.
- The two categories can be separated by using a linear function (aka hyperplane) defined by:

$$\hat{y}_n = \mathbf{w}^T \mathbf{x}_n + b, \quad \hat{y}_n \in \{1, -1\}$$

where $n \in \{1, \dots, N\}$

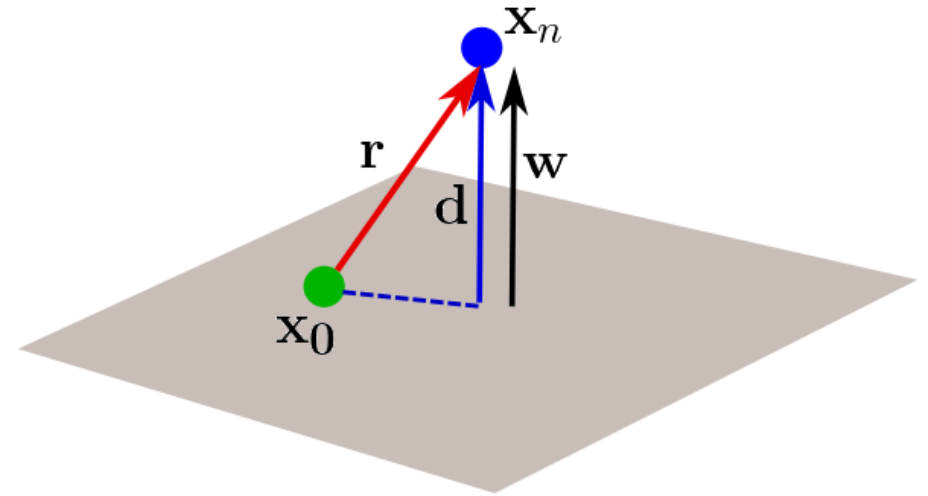
$\mathbf{x}_n$ denotes a data point,
 $\mathbf{w}$ is a vector of weights,
 b is the bias,
 $\mathbf{y}_n$ is the model's prediction

- In this case each prediction can be either 1 or -1 for points that lie on both sides of the hyperplane and 0 for points that lie on the hyperplane itself.
- **Goal:** Given a training dataset comprising of several $\mathbf{x}$'s with their corresponding $\mathbf{y}$'s, the goal is to find values for $\mathbf{w}$ and b that give us the best possible margin.

Deriving the primal problem

- $\mathbf{x}_n$ is one of the points closest to the hyperplane and it creates an orthogonal vector $\mathbf{d}$ that stems from it.
- vector $\mathbf{d}$ is in the same direction as $\mathbf{w}$.
- Any point $\mathbf{x}_0$ that lies on the hyperplane will form a vector $\mathbf{r}$ with $\mathbf{x}_n$.
- $\mathbf{d}$ is the projection of $\mathbf{r}$ on $\mathbf{w}$ given by:

$$\mathbf{d} = \frac{\mathbf{w}}{\|\mathbf{w}\|} \mathbf{r} = \frac{\mathbf{w}^\top (\mathbf{x}_n - \mathbf{x}_0)}{\|\mathbf{w}\|} = \frac{\mathbf{w}^\top \mathbf{x}_n - \mathbf{w}^\top \mathbf{x}_0}{\|\mathbf{w}\|}$$



Geometric representation of variables

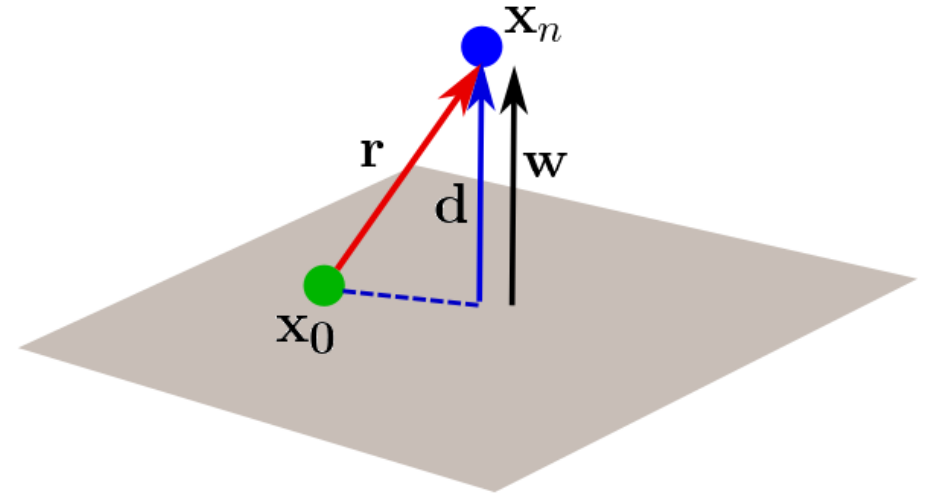
Deriving the primal problem

$$\mathbf{d} = \frac{\mathbf{w}}{\|\mathbf{w}\|} \mathbf{r} = \frac{\mathbf{w}^\top (\mathbf{x}_n - \mathbf{x}_0)}{\|\mathbf{w}\|} = \frac{\mathbf{w}^\top \mathbf{x}_n - \mathbf{w}^\top \mathbf{x}_0}{\|\mathbf{w}\|}$$

- add b and $-b$ to the numerator.
- The magnitude of $\mathbf{d}$ gives the distance of $\mathbf{x}$ to the hyperplane, and is defined by:

$$d = \frac{|\mathbf{w}^\top \mathbf{x}_n + b - (\mathbf{w}^\top \mathbf{x}_0 + b)|}{\|\mathbf{w}\|} = \frac{|\mathbf{w}^\top \mathbf{x}_n + b|}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|}$$

- Note: since $\mathbf{x}_0$ lies on the hyperplane, the corresponding prediction $\mathbf{y}_0$ is zero



Geometric representation of variables

Deriving the primal problem

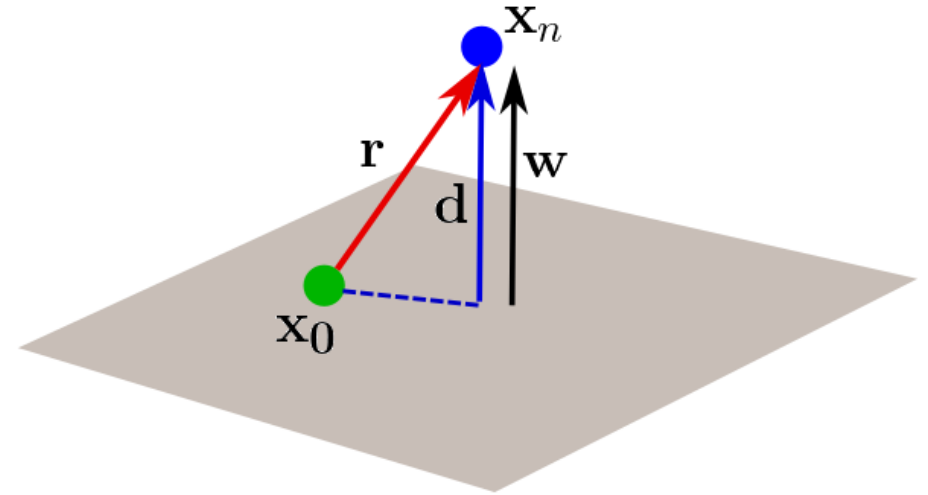
- **Objective:** to find the optimal margin, meaning to maximize d .
- The optimization problem is:

$$d = \max \left(\frac{1}{\|\mathbf{w}\|} \right) \quad \text{subject to} \quad \min_n |\mathbf{w}^\top \mathbf{x}_n + b| = 1$$

- Let's turn it into a minimization problem:

$$d = \min \left(\frac{1}{2} \|\mathbf{w}\|^2 \right) = \min \left(\frac{1}{2} \mathbf{w}^\top \mathbf{w} \right)$$

Why raise $\mathbf{w}$ to the power 2 and then divide it by 2? a mathematical trick to make things easier, since the derivative for this expression is simply $\mathbf{w}$. Moreover, a quadratic programming problem is easier to be solved using popular optimization libraries.



Geometric representation of variables

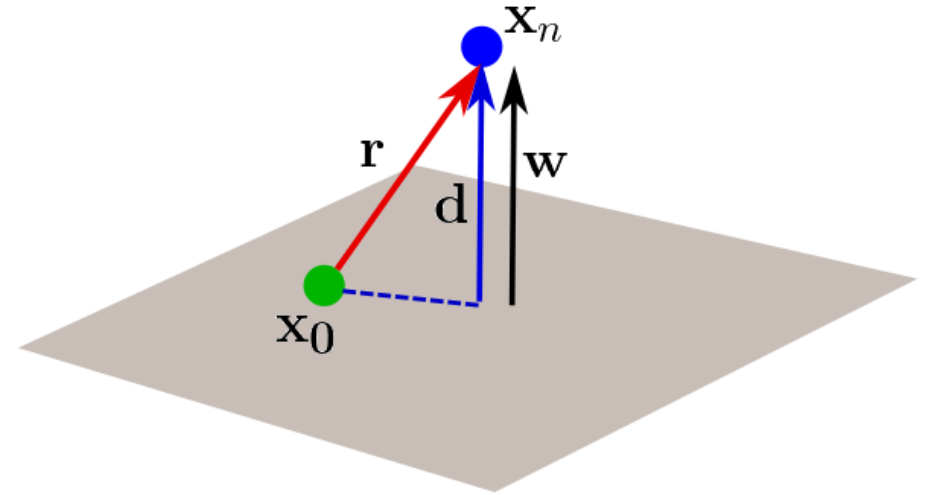
Deriving the primal problem

- Now, let's look at the constraint:

$$\min_n |\mathbf{w}^\top \mathbf{x}_n + b| = 1$$

- How can we get rid of absolute value?
- We want the model to classify all points correctly. How to define it in mathematical terms?

$$y_n : \text{Given answer} \in \{-1, 1\}$$
$$\hat{y}_n : \text{Predicted answer} \in \{-1, 1\}$$

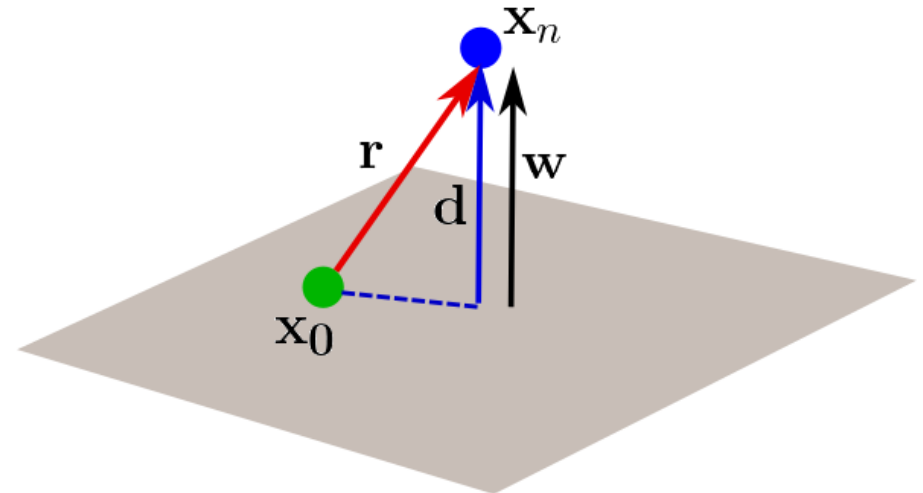


Geometric representation of variables

Deriving the primal problem

- The model should make predictions that are equal to the correct answer y_n .
- This happens when the given and predicted answers have the same sign, the multiplication of both is at least one.

$$y_n \hat{y}_n \geq 1 \implies y_n (\mathbf{w}^\top \mathbf{x}_n + b) \geq 1$$



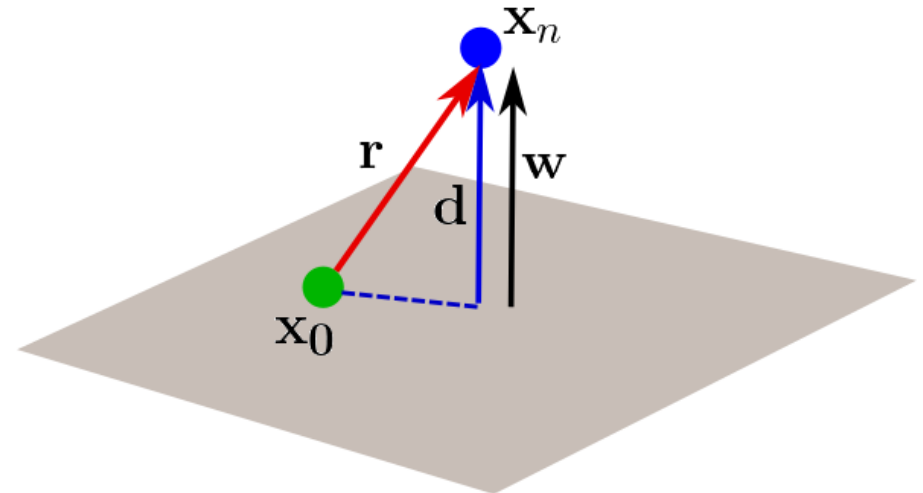
Geometric representation of variables

Deriving the primal problem

- The optimization problem will be:

$$L(\mathbf{w}, b, \alpha) = \frac{1}{2} \mathbf{w}^T \mathbf{w} - \sum_{n=1}^N \alpha_n [y_n (\mathbf{w}^T \mathbf{x}_n + b) - 1]$$

- Note: for each $(\mathbf{x}_n, y_n)$ there is an associated Lagrange multiplier α .
- The data points $\mathbf{x}_n$ whose alpha value is greater than zero are known as *support vectors*, since they influence the behavior of the separating hyperplane.
- Challenge: this problem depends on three variables.



Geometric representation of variables

Deriving the primal problem

- **Challenge:** this problem depends on three variables.

$$L(\mathbf{w}, b, \alpha) = \frac{1}{2} \mathbf{w}^\top \mathbf{w} - \sum_{n=1}^N \alpha_n [y_n (\mathbf{w}^\top \mathbf{x}_n + b) - 1] \quad (1)$$

- Let's simplify it.

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum_{n=1}^N \alpha_n y_n \mathbf{x}_n = 0, \quad \frac{\partial L}{\partial b} = - \sum_{n=1}^N \alpha_n y_n = 0$$

$$\mathbf{w} = \sum_{n=1}^N \alpha_n y_n \mathbf{x}_n, \quad \sum_{n=1}^N \alpha_n y_n = 0$$

- Replacing in Equ. (1)

$$L(\alpha) = \frac{1}{2} \left(\sum_{m=1}^N \alpha_m y_m \mathbf{x}_m \right) \left(\sum_{n=1}^N \alpha_n y_n \mathbf{x}_n \right) - \left(\sum_{n=1}^N \alpha_n y_n \mathbf{x}_n \right) \left(\sum_{m=1}^N \alpha_m y_m \mathbf{x}_m \right) + \sum_{n=1}^N \alpha_n$$

Rearranging leads to: $\longrightarrow$
$$L(\alpha) = \sum_{n=1}^N \alpha_n - \frac{1}{2} \sum_{m=1}^N \sum_{n=1}^N \alpha_m \alpha_n y_m y_n \mathbf{x}_m \mathbf{x}_n$$

Summary of Linear SVM

- Objective: Maximize the distance between these two hyperplanes.
- Max $\frac{2}{\|w\|}$
- Subject to:

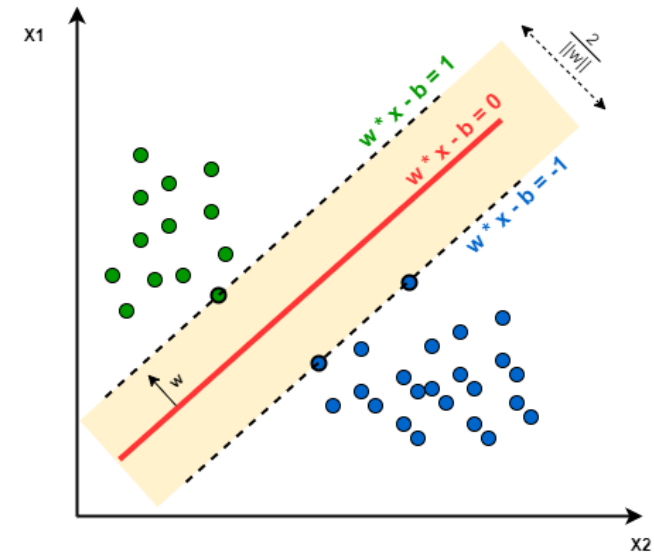
$$w^T x_i - b \geq 1, \text{ if } y_i = 1,$$

or

$$w^T x_i - b \leq -1, \text{ if } y_i = -1.$$

- The constraints can be rewritten as:

$$y_i (w^T x_i - b) \geq 1, \quad \text{for all } 1 \leq i \leq n.$$



Ref:
<https://www.analyticsvidhya.com/blog/2021/05/multiclass-classification-using-svm/>

SVM vs. Logistic Regression

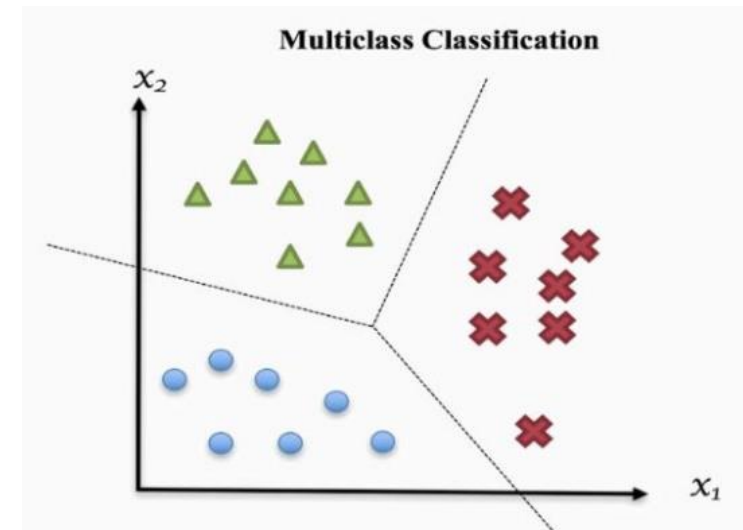
- In logistic regression, we take the output of the linear function and squeeze the value within the range of $[0,1]$ using the sigmoid function. If the squeezed value is greater than a threshold value (0.5) we assign it a label 1, otherwise 0.
- In SVM, if that output is greater than 1, we identify it with one class and if the output is -1, we identify it with another class. Since the threshold values are changed to 1 and -1 in SVM, we obtain this reinforcement range of values $[-1,1]$ which acts as margin.

Are SVMs only for binary classification?

- SVM's are initially meant to perform binary classification.
- SVM can be extended to multi-class classification.
- One approach is to create several one-vs-all classifiers.
- In general, any binary classification can be extended to multi-class case by using one-vs-all method.
- In other words, instead of working with one classification for N classes you deal with N binary classification of given class vs the rest of classes.

Multi-class Classification

- **Multi-class classification:** A technique that helps categorize the test data into multiple class labels present in trained data as a model prediction.
- Two types of multi-class classification:
 - One vs. All (one-vs-rest)
 - One vs. One

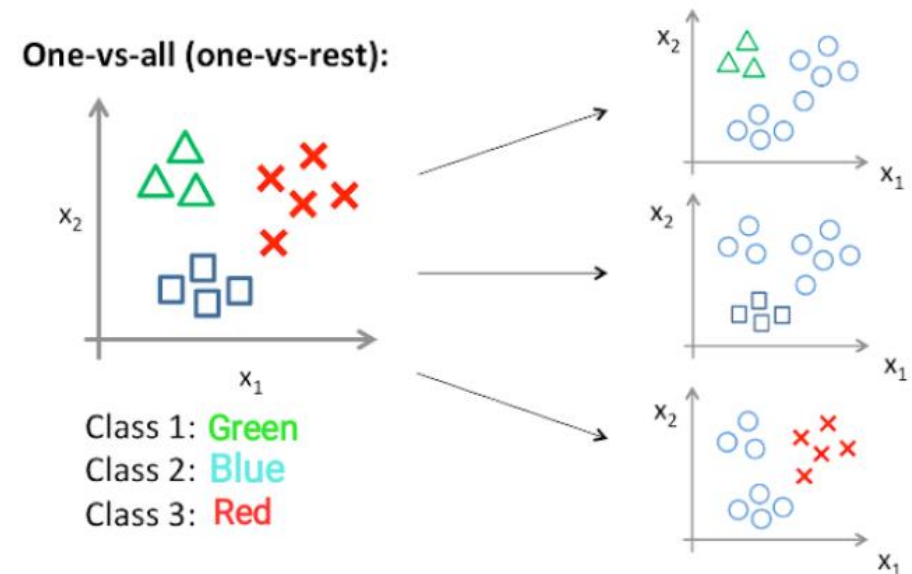


The Number of Classifier Models

- **How many classifier models we need for multi-class classification?**
The number of classifier models depends on the type of classification technique.
- **Types of multi-class classification:**
 - One vs. All:- **N -class instances then N binary classifier models** are needed.
 - One vs. One:- **N -class instances then $N*(N-1)/2$ binary classifier models**

One vs. All (One-vs-Rest)

- For the N-class instances dataset, N-binary classifier models should be generated.
- The number of generated binary classifiers is the same as the number of class labels present in the dataset.



Example of One vs. All

- Three classes: type 1 for Green, type 2 for Blue, and type 3 for Red.
- We should create three classifiers.

- **Classifier 1:- [Green] vs [Red, Blue]**
- **Classifier 2:- [Blue] vs [Green, Red]**
- **Classifier 3:- [Red] vs [Blue, Green]**

Class 1 :- Green

Class 2 :- Blue

Class 3 :- Red

Features			Classes
x1	x2	x3	G
x4	x5	x6	B
x7	x8	x9	R
x10	x11	x12	G
x13	x14	x15	B
x16	x17	x18	R

Example of One vs. All

- We should create a training dataset for each class.

Main Dataset

Features			Classes
x1	x2	x3	G
x4	x5	x6	B
x7	x8	x9	R
x10	x11	x12	G
x13	x14	x15	B
x16	x17	x18	R

Class 1 :- Green Class 2 :- Blue Class 3 :- Red

Training Dataset 1
Class :- Green

Features			Green
x1	x2	x3	+1
x4	x5	x6	-1
x7	x8	x9	-1
x10	x11	x12	+1
x13	x14	x15	-1
x16	x17	x18	-1

Training dataset for Green class

Example of One vs. All

- We should create a training dataset for each class.

Training Dataset 2
Class :- Blue

Features			Blue
x1	x2	x3	-1
x4	x5	x6	+1
x7	x8	x9	-1
x10	x11	x12	-1
x13	x14	x15	+1
x16	x17	x18	-1

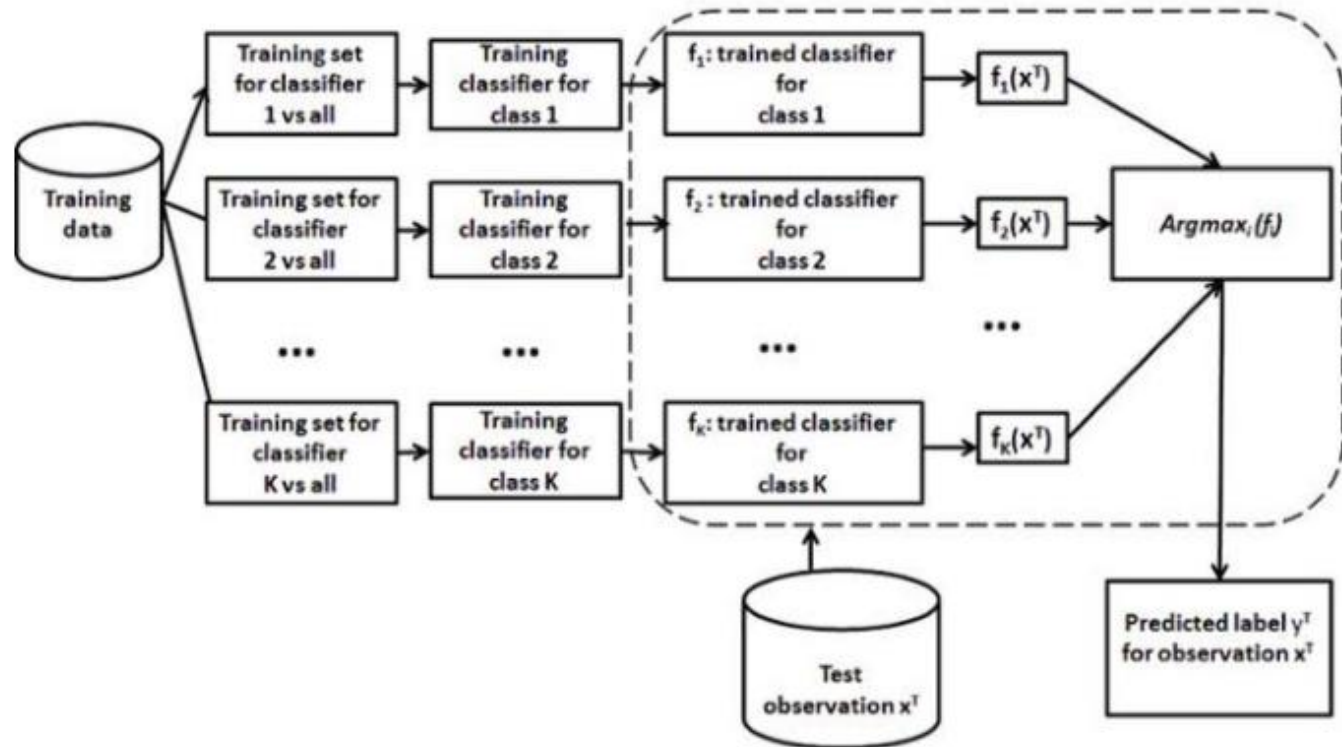
Training Dataset 3
Class :- Red

Features			Red
x1	x2	x3	-1
x4	x5	x6	-1
x7	x8	x9	+1
x10	x11	x12	-1
x13	x14	x15	-1
x16	x17	x18	+1

Training dataset for Blue class and Red class

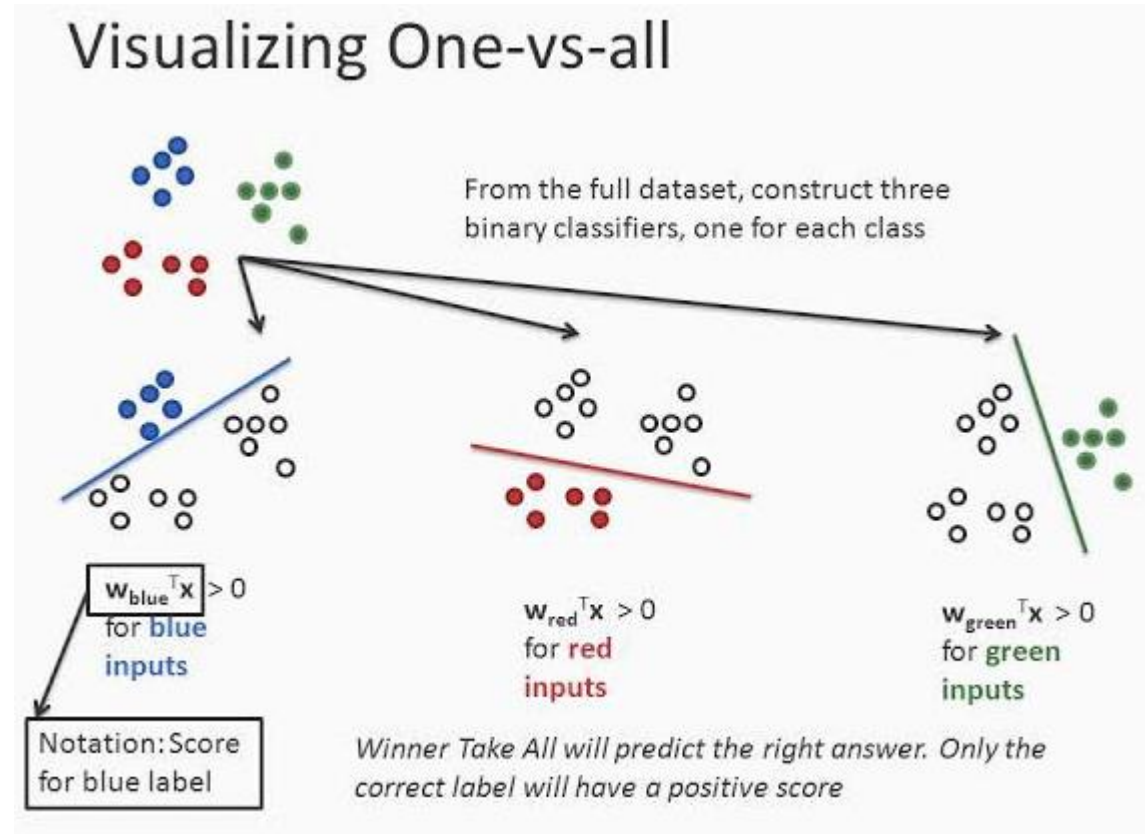
Example of One vs. All

- After a training dataset is created for each classifier, the info is provided to the classifier model.
- By analyzing the probability scores, the result is predicted as the class index having a maximum probability score.



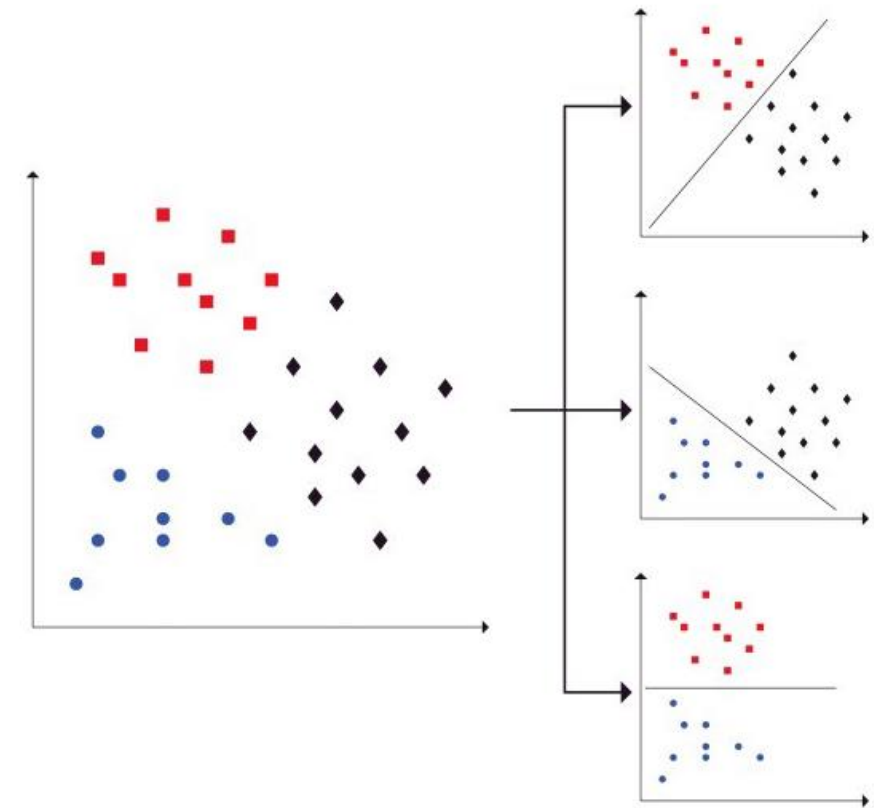
Example of One vs. All

- The **Green** class classifier with a probability score of **(0.9)**.
- The **Blue** class with a probability score of **(0.4)**.
- Then the test input belongs to the **Green** class.



One vs. One

- For the **N-class** instances dataset, we should generate the $N * (N-1)/2$ binary classifier models.
- Example: three types: **Green**, **Blue**, and **Red** ($N=3$).
- This problem is divided into $N * (N-1)/2 = 3$ binary classifier problems:
 - Classifier 1: Green vs. Blue
 - Classifier 2: Green vs. Red
 - Classifier 3: Blue vs. Red



Evaluation Metrics for Classification

- Evaluation metrics explain the performance of a model.
- How accurate is this model?

Evaluation Metrics for Classification

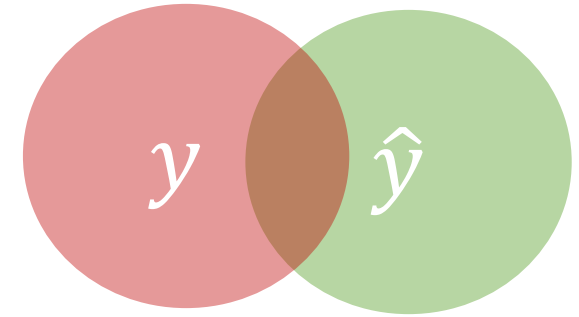
- Jaccard Index
- Confusion Matrix
- F1-score
- Log loss

Jaccard Index

- $\hat{y}$: Predicted values
- y : Actual values

$$j(y, \hat{y}) = \frac{|y \cap \hat{y}|}{|y \cup \hat{y}|}$$

$$= \frac{|y \cap \hat{y}|}{|y| + |\hat{y}| - |y \cap \hat{y}|}$$



- Example:

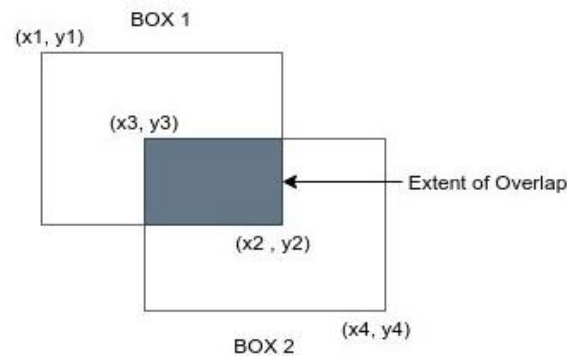
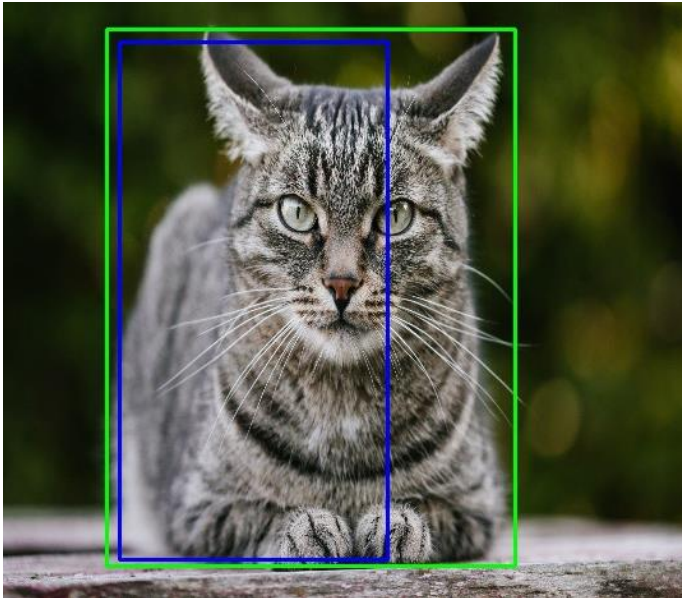
predicted : [0, 0, 0, 0, 0, 1, 1, 1, 1, 1]

actual : [1, 1, 0, 0, 0, 1, 1, 1, 1, 1]

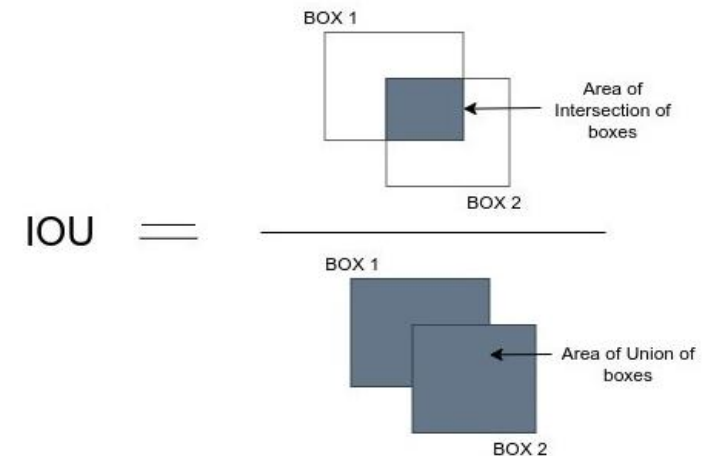
$$j(y, \hat{y}) = \frac{8}{10 + 10 - 8} = 0.66$$

Jaccard Index

- IOU (Intersection over Union): define the extent of overlap of two boxes.



$$IOU = \frac{\text{Area of Intersection of two boxes}}{\text{Area of Union of two boxes}}$$



Jaccard Index

- The greater the overlap, the greater the IOU.



Confusion Matrix

1. True positive (TP):

The model *correctly* predicted *Positive* cases as *Positive*.

2. False positive (FP):

The model *incorrectly* predicted *Negative* cases as *Positive*. (Type I error)

3. False Negative (FN):

The model *incorrectly* predicted *Positive* cases as *Negative* (Type II error)

4. True Negative (TN):

The model *correctly* predicted *Negative* cases as *Negative*.

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Confusion Matrix

- Example:

Actual values = ['dog', 'cat', 'dog', 'cat', 'dog', 'dog', 'cat', 'dog', 'cat', 'dog']

Predicted values = ['dog', 'dog', 'dog', 'cat', 'dog', 'dog', 'cat', 'cat', 'cat', 'cat']

		PREDICTIVE VALUES	
		POSITIVE (CAT)	NEGATIVE (DOG)
ACTUAL VALUES	POSITIVE (CAT)	TRUE POSITIVE 3 YOU ARE A CAT.	FALSE NEGATIVE 1 YOU ARE A DOG. TYPE II ERROR
	NEGATIVE (DOG)	FALSE POSITIVE 2 YOU ARE A CAT. TYPE I ERROR	TRUE NEGATIVE 4 YOU ARE NOT A CAT.

F1-Score

- Using the confusion matrix, we can calculate the **accuracy, precision** and the **recall** scores.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

F1-Score

- **Precision:** the ratio of the total number of correctly classified positive classes divided by the total number of predicted positive classes.
- Or, out of all the predictive positive classes, how many were predicted correctly.
- Precision should be high.

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

$$\text{Precision} = \frac{TP}{TP + FP} \text{ or } \frac{\text{True Positive}}{\text{Predictive Results}}$$

F1-Score

- **Recall (sensitivity):** the ratio of the total number of correctly classified positive classes divide by the total number of positive classes.
- Or, out of all the positive classes, how much we have predicted correctly.
- Recall should be high.

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

$$\text{Recall} = \frac{TP}{TP + FN} \text{ or } \frac{\text{True Positive}}{\text{Actual Results}}$$

F1-Score

- **Specificity:** the proportion of actual negatives that are correctly identified.

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

F1-score or F-score

- It is difficult to compare two models with different Precision and Recall. So, F-Score can be used to compare them.

- F-score is the Harmonic Mean of Precision and Recall. $H = \frac{n}{\frac{1}{x_1} + \frac{1}{x_2} + \frac{1}{x_3} + \dots + \frac{1}{x_n}}$
- F-score should be high.

$$F - score = \frac{2 * Recall * Precision}{Recall + Precision}$$

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Log loss

- Is used in cases where the outcome of the classifier is a class probability and not a class label (e.g., logistic regression).
- Loss log equation can be used for each row in the data set.

$$\text{Logloss}_i = -[y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$$

- An average of the log loss from all the rows gives the value of the log loss.

$$\text{Logloss} = \frac{1}{N} \sum_{i=1}^N \text{logloss}_i$$

$$\text{Logloss} = -\frac{1}{N} \sum_{i=1}^N [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$$

i is the given observation/record,
 y is the actual/true value,
 p is the prediction probability

Confusion matrix for Multi-class classification

- In the multi-class classification, TP, TN, FP, FN values are not given directly as in the binary classification problem.
- We need to calculate them for each class.



		Predicted Values		
		Setosa	Versicolor	Virginica
Actual Values	Setosa	16 (cell 1)	0 (cell 2)	0 (cell 3)
	Versicolor	0 (cell 4)	17 (cell 5)	1 (cell 6)
	Virginica	0 (cell 7)	0 (cell 8)	11 (cell 9)

Confusion matrix for Multi-class classification

- In the multi-class classification, TP, TN, FP, FN values are not given directly as in the binary classification problem.
- We need to calculate them for each class.
- Example: calculate the TP, TN, FP, FN values for the class Setosa:



		Predicted Values		
		Setosa	Versicolor	Virginica
Actual Values	Setosa	16 (cell 1)	0 (cell 2)	0 (cell 3)
	Versicolor	0 (cell 4)	17 (cell 5)	1 (cell 6)
	Virginica	0 (cell 7)	0 (cell 8)	11 (cell 9)

Confusion matrix for Multi-class classification

- **TP:** The actual value and predicted value should be the same (cell 1).
- **FN:** The sum of values of corresponding rows except the TP value (cell 2 + cell 3) = $(0 + 0) = 0$
- **FP:** The sum of values of corresponding column except the TP value. (cell 4 + cell 7) = $(0 + 0) = 0$
- **TN:** The sum of values of all columns and row except the values of that class that we are calculating the values for. (cell 5 + cell 6 + cell 8 + cell 9) = $17 + 1 + 0 + 11 = 29$

		Predicted Values		
		Setosa	Versicolor	Virginica
Actual Values	Setosa	16 (cell 1)	0 (cell 2)	0 (cell 3)
	Versicolor	0 (cell 4)	17 (cell 5)	1 (cell 6)
	Virginica	0 (cell 7)	0 (cell 8)	11 (cell 9)

References

- <https://www.analyticsvidhya.com/blog/2020/06/4-ways-split-decision-tree/>
- <https://medium.com/swlh/decision-tree-classification-de64fc4d5aac>
- <https://towardsdatascience.com/entropy-how-decision-trees-make-decisions-2946b9c18c8>